

Cross Validation on a Real Race

Cole Cappello

09-29-25

```
# Setup
library(tidyverse)
library(Matrix)
library(rstan)

options(mc.cores = parallel::detectCores())
```

The Hungarian Grand Prix 2025

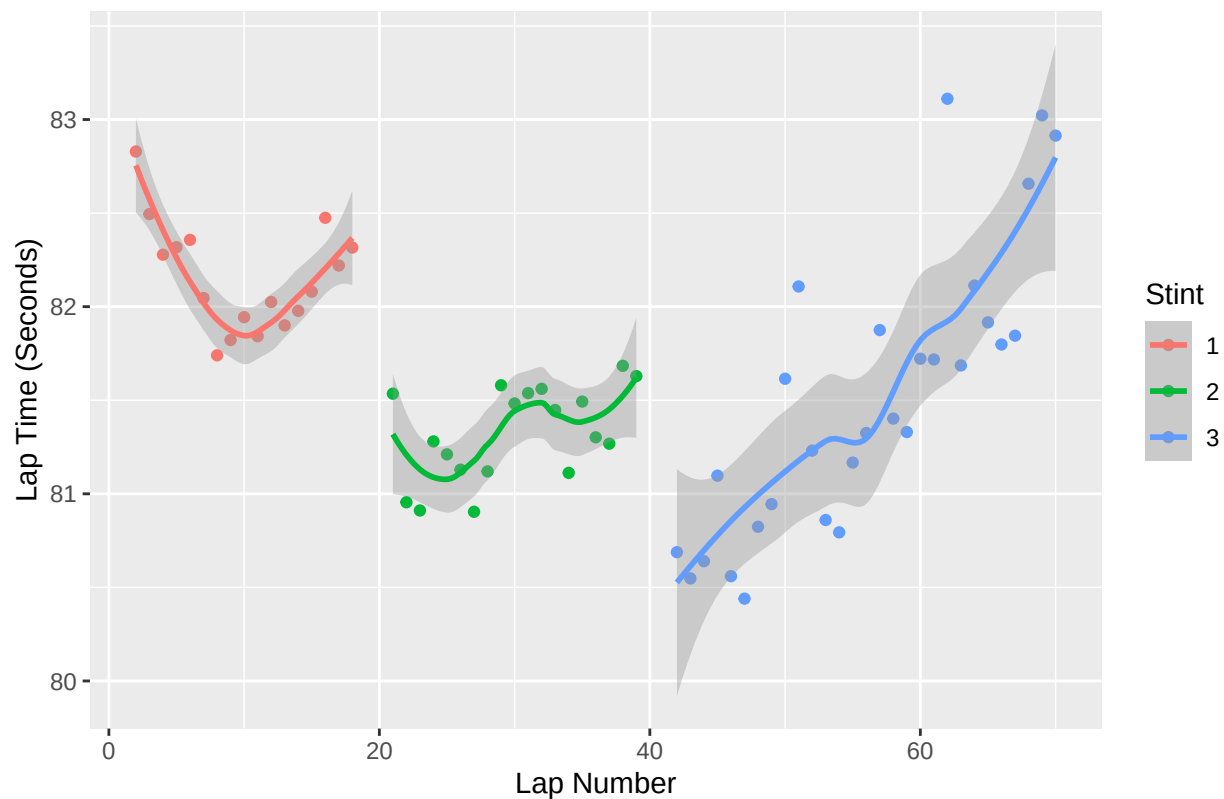
I'm going to load lap time data from the Hungarian Grand Prix for various drivers. We'll do cross validation on Charles Leclerc's race because - while I admit I am slightly partial - he does seem to have a race that conforms relatively well to our model. Here is a visual of Leclerc's lap times:

```
hungary_laps <- read_csv("Hungary_laps.csv")

hungary_laps <- hungary_laps %>%
  mutate(Stint = as.factor(Stint),
         Compound = as.factor(Compound))

filter(hungary_laps, Driver == "LEC", is.na(PitOutTime) & is.na(PitInTime), LapTime < 100) %>%
  ggplot(mapping = aes(x = LapNumber, y = LapTime, colour = Stint)) +
  geom_point() +
  geom_smooth() +
  labs(title = "Tire Degradation -- Charles Leclerc (Ferrari)",
       x = "Lap Number",
       y = "Lap Time (Seconds)")
```

Tire Degration -- Charles Leclerc (Ferrari)



Cross Validation Code

In this section I will include the code used for cross validation. Skip to the next section if interested in results.

Stan CV code

```
stint_cv <- function(model_string, data_list) {  
  
  # Store the full data/time series  
  y <- data_list[["y"]]  
  # Store the full compound vector  
  compound <- data_list[["Compound"]]  
  # Store the full pit vector  
  pit <- data_list[["Pit"]]  
  # Store the Fuel vector  
  fuel <- data_list[["fuel_mass"]]  
  
  # Get the index of laps that end a stint  
  end_stint <- which(data_list[["Pit"]] == 1)  
  # Add the last lap because that also marks the end of a stint  
  end_stint <- c(end_stint, length(y))  
  
  # Number of Stints  
  num_stints <- length(end_stint)
```

```

# Will get RMSPE for each stint so need an empty vector
RMSPE_vec <- rep(NA, num_stints)

# Will get plots for each stint
plots <- vector("list", length = num_stints)

for(j in 1:num_stints) {

  # Do CV on last quarter of each stint
  # Number of folds
  if(j == 1) {
    K <- round(end_stint[j]/4)
  } else {
    K <- round((end_stint[j] - end_stint[j-1])/4)
  }

  stan.models <- vector("list", length = K)
  posteriors <- vector("list", length = K)

  # Vector to Store one-step ahead predictions
  ypred_hat <- rep(NA, K)

  for(i in 1:K) {

    # Update data vector to allow for next step prediction
    data_list[["TT"]] <- length(y[1:(i+end_stint[j]-K-1)])
    data_list[["y"]] <- y[1:(i+end_stint[j]-K-1)]
    data_list[["Compound"]] <- compound[1:(i+end_stint[j]-K-1)]
    c_map <- sort(unique(compound[1:(i+end_stint[j]-K-1)]))
    data_list[["compound_map"]] <- array(c_map, dim = length(c_map))
    data_list[["C_used"]] <- length(c_map)
    data_list[["Pit"]] <- pit[1:(i+end_stint[j]-K-1)]
    data_list[["fuel_mass"]] <- fuel[1:(i+end_stint[j]-K-1)]

    # Fit model on subset of data then get one step ahead prediction
    stan.models[[i]] <- stan(file = model_string,
                           data = data_list,
                           chains = 3, iter = 30000,
                           control = list(adapt_delta = .99, max_treedepth = 12))
    posteriors[[i]] <- extract(stan.models[[i]])
    ypred_hat[i] <- mean(posteriors[[i]]$y_pred)

  }

  # Plots
  df <- tibble(laps = 1:end_stint[j],
               y = y[1:end_stint[j]])

  pred_df <- tibble(time = (end_stint[j] - K + 1):end_stint[j],
                   pred = ypred_hat)

  plots[[j]] <- ggplot() +
    geom_point(data = df,

```

```

        aes(x = laps, y = y, color = "Observations")) +
geom_point(data = pred_df,
        aes(x = time, y = pred, color = "Predictions")) +
scale_color_manual(values = c("Observations" = "red",
                             "Predictions" = "blue")) +
labs(title = paste("Scatter Plot of Lap vs Lap Times with Predictions for Stint", j),
     x = "Lap",
     y = "Laptime in Seconds",
     color = "Legend")

# Calculate Summary Statistic
MSPE <- mean((y[(end_stint[j]-K+1):end_stint[j]] - ypred_hat)^2)
RMSPE_vec[j] <- sqrt(MSPE)

}
return(list(RMSPE_vec, plots))
}

```

Arima Baseline CV code

```

arima_cv <- function(y, pit) {

  # Get the index of laps that end a stint
  end_stint <- which(pit == 1)
  # Add the last lap because that also marks the end of a stint
  end_stint <- c(end_stint, length(y))

  # Number of Stints
  num_stints <- length(end_stint)

  # Will get RMSPE for each stint so need an empty vector
  RMSPE_vec <- rep(NA, num_stints)

  # Will get plots for each stint
  plots <- vector("list", length = num_stints)

  for(i in 1:num_stints) {

    # Number of folds
    if(i == 1) {
      K <- round(end_stint[i]/4)
    } else {
      K <- round((end_stint[i] - end_stint[i-1])/4)
    }

    arma.models <- vector("list", length = K)

    # Vector to Store one-step ahead predictions
    ypred_hat <- rep(NA, K)

    for(j in 1:K) {

      arma.models[[j]] <- arima(y[1:(j+end_stint[i]-K-1)], order = c(2,1,2))
    }
  }
}

```

```

    ypred_hat[j] <- as.numeric(predict(arma.models[[j]],n.ahead = 1)$pred)
  }
  # Plots
  df <- tibble(laps = 1:end_stint[i],
              y = y[1:end_stint[i]])

  pred_df <- tibble(time = (end_stint[i] - K + 1):end_stint[i],
                  pred = ypred_hat)

  plots[[i]] <- ggplot() +
    geom_point(data = df,
              aes(x = laps, y = y, color = "Observations")) +
    geom_point(data = pred_df,
              aes(x = time, y = pred, color = "Predictions")) +
    scale_color_manual(values = c("Observations" = "red",
                                "Predictions" = "blue")) +
    labs(title = paste("Scatter Plot of Lap vs Lap Times with Predictions for Stint", i),
         x = "Lap",
         y = "Laptime in Seconds",
         color = "Legend")

  # Calculate Summary Statistic
  MSPE <- mean((y[(end_stint[i]-K+1):end_stint[i]] - ypred_hat)^2)
  RMSPE_vec[i] <- sqrt(MSPE)
}

return(list(RMSPE_vec,plots))
}

```

Analysis

Get the data ready

```

leclerc_df <- hungary_laps %>%
  filter(Driver == "LEC", is.na(PitOutTime) & is.na(PitInTime), LapTime < 100) %>%
  mutate(Compound_Code = as.integer(factor(Compound,levels = c("HARD","MEDIUM","SOFT"))))

leclerc_laps <- leclerc_df$LapTime

leclerc_compound <- leclerc_df$Compound_Code

leclerc_pit <- rep(0,length(leclerc_compound))

for(i in 2:length(leclerc_df$Stint)) {
  if(leclerc_df$Stint[i] != leclerc_df$Stint[i-1]) {
    leclerc_pit[i-1] <- 1
  }
}

fuel.kg <- seq(110,1,length.out = length(leclerc_laps))

```

```
leclerc_map <- sort(unique(leclerc_compound))
leclerc_used <- length(leclerc_map)
```

Cross Validation Results

```
# Arima cv
arima_results <- arima_cv(leclerc_laps, leclerc_pit)

# Stan SSM CV
leclerc_data <- list(TT = length(leclerc_laps), y = leclerc_laps, C = 3, Compound = leclerc_compound, C)

stan_results <- stint_CV("full_race_1driver_and_fuel_real.stan", leclerc_data)

# Stint 1
arima_results[[1]][[1]]

## [1] 0.2807924
stan_results[[1]][[1]]

## [1] 0.2327326

# Stint 2
arima_results[[1]][[2]]

## [1] 0.3925258
stan_results[[1]][[2]]

## [1] 0.2163307

# Stint 3
arima_results[[1]][[3]]

## [1] 0.4572783
stan_results[[1]][[3]]

## [1] 0.4051673
```

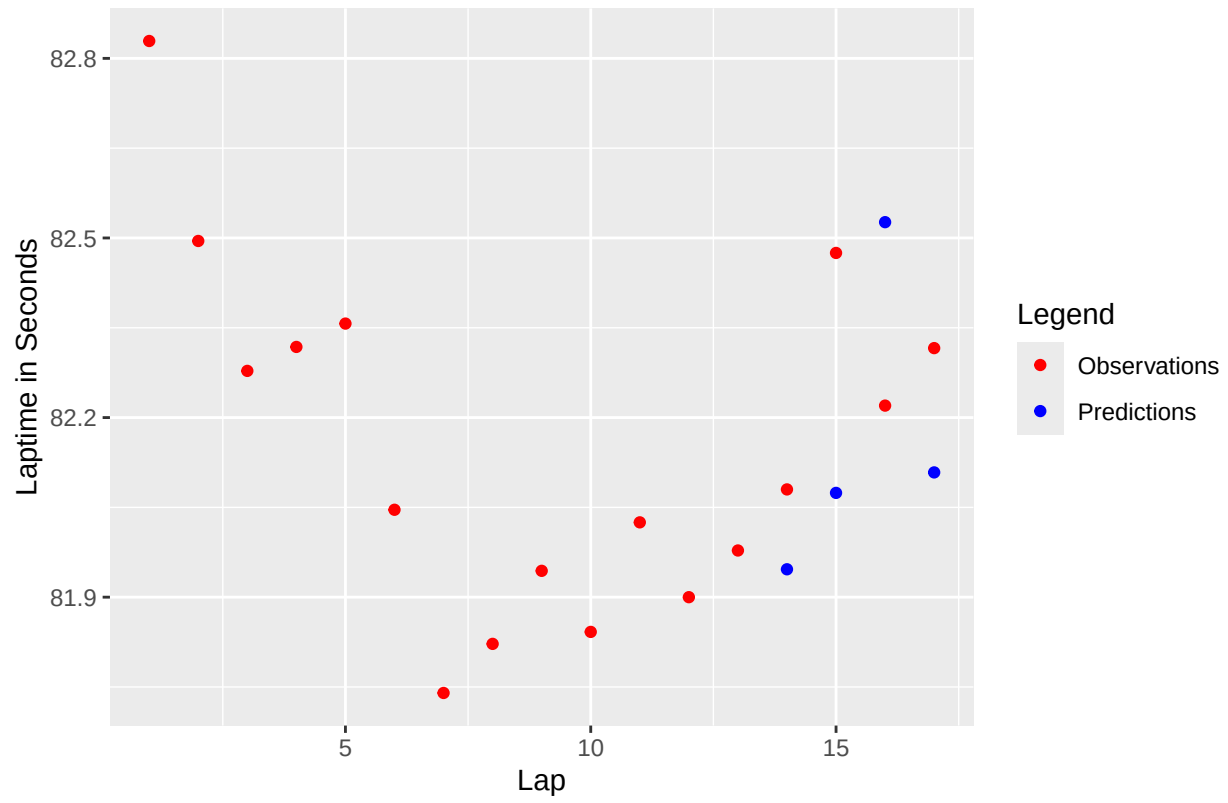
Appendix

Plots of lap time vs lap with observations and predictions from both the arima model and our full SSM.

Arima Predictions

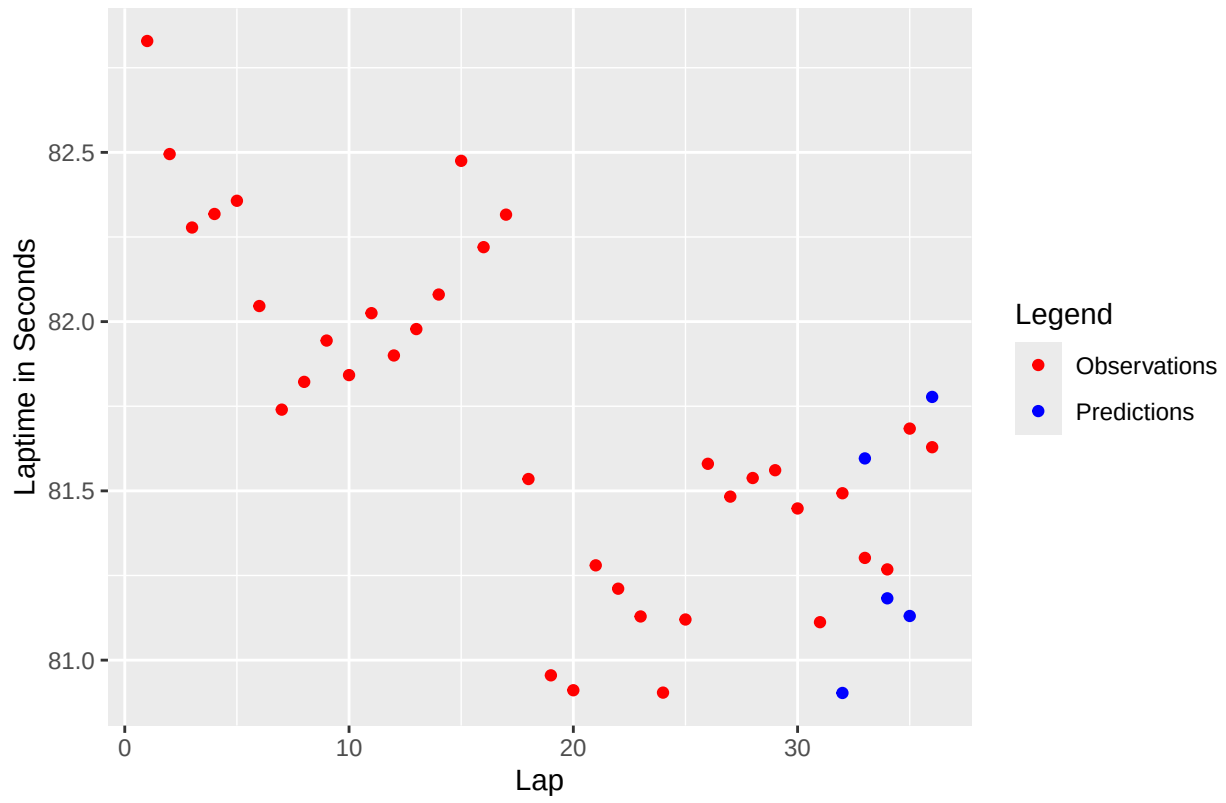
```
arima_results[[2]][[1]]
```

Scatter Plot of Lap vs Lap Times with Predictions for Stint 1

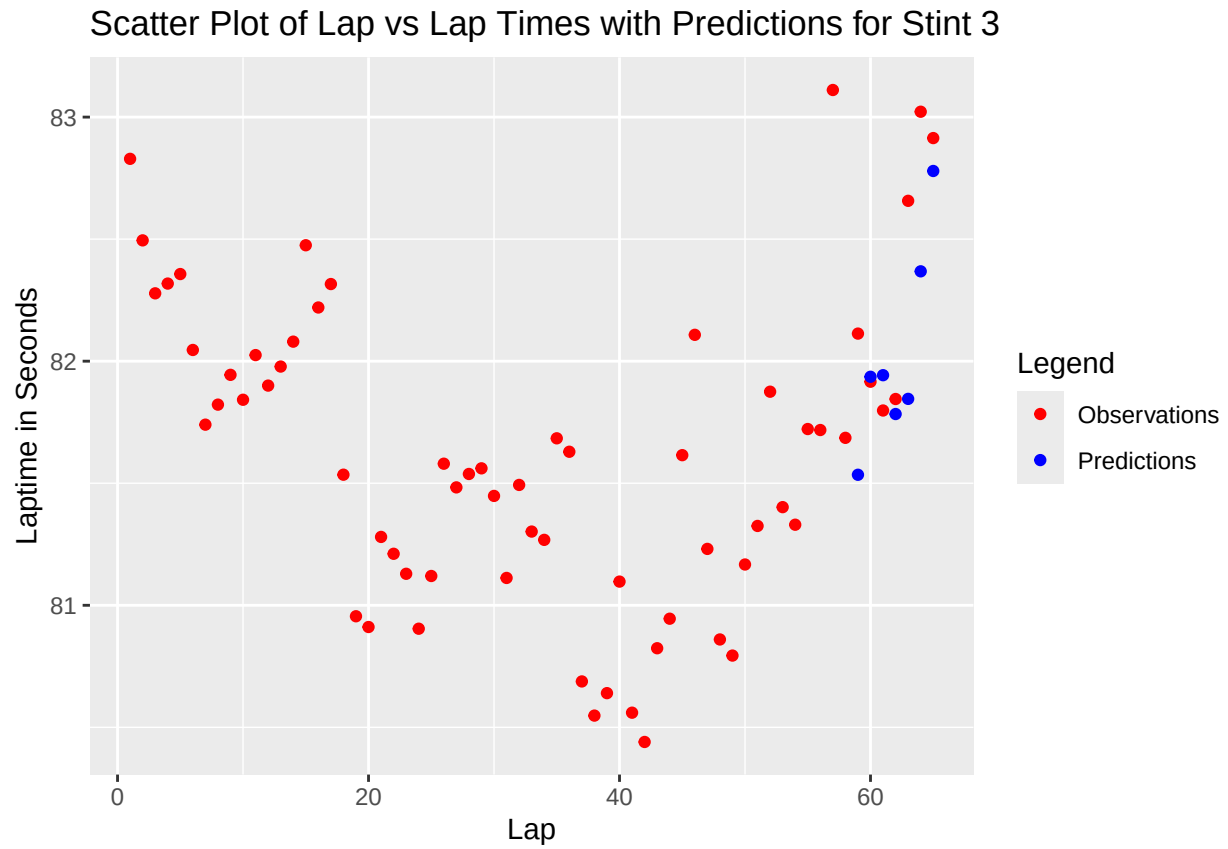


```
arima_results[[2]][[2]]
```

Scatter Plot of Lap vs Lap Times with Predictions for Stint 2



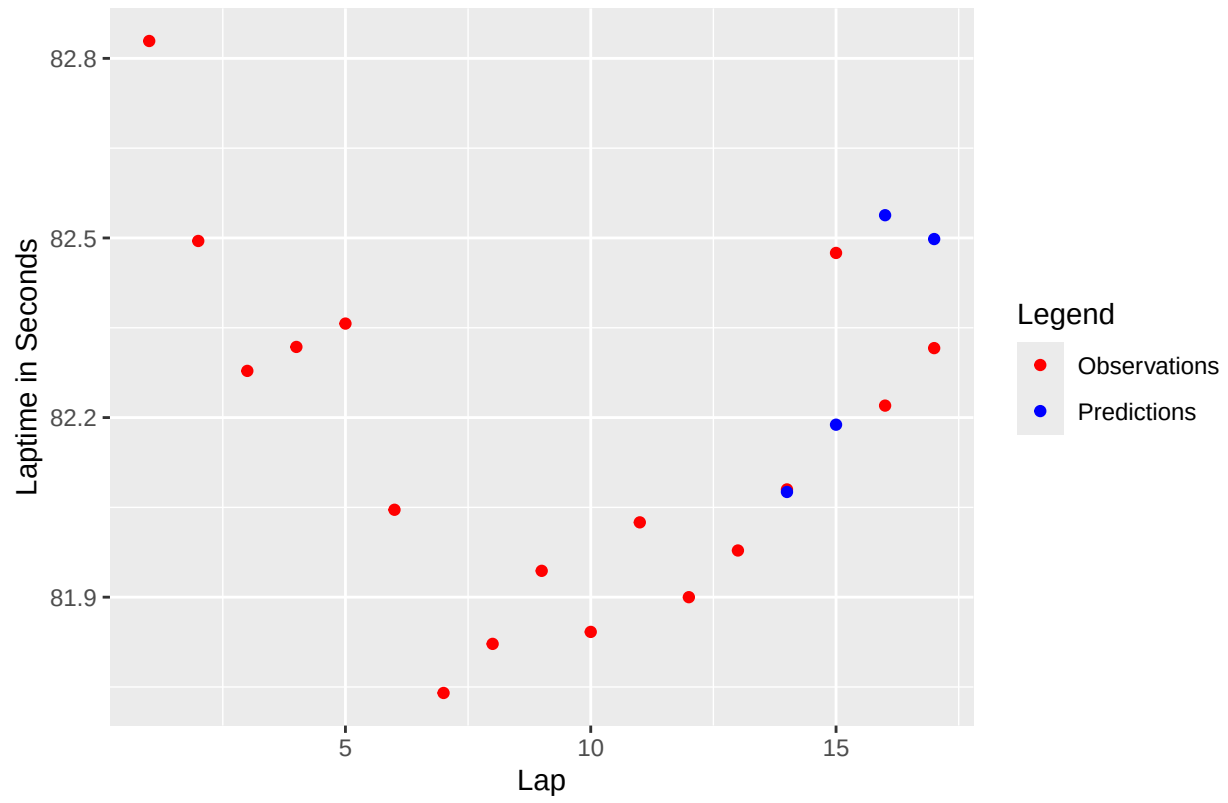
```
arima_results[[2]][[3]]
```

SSM Predictions

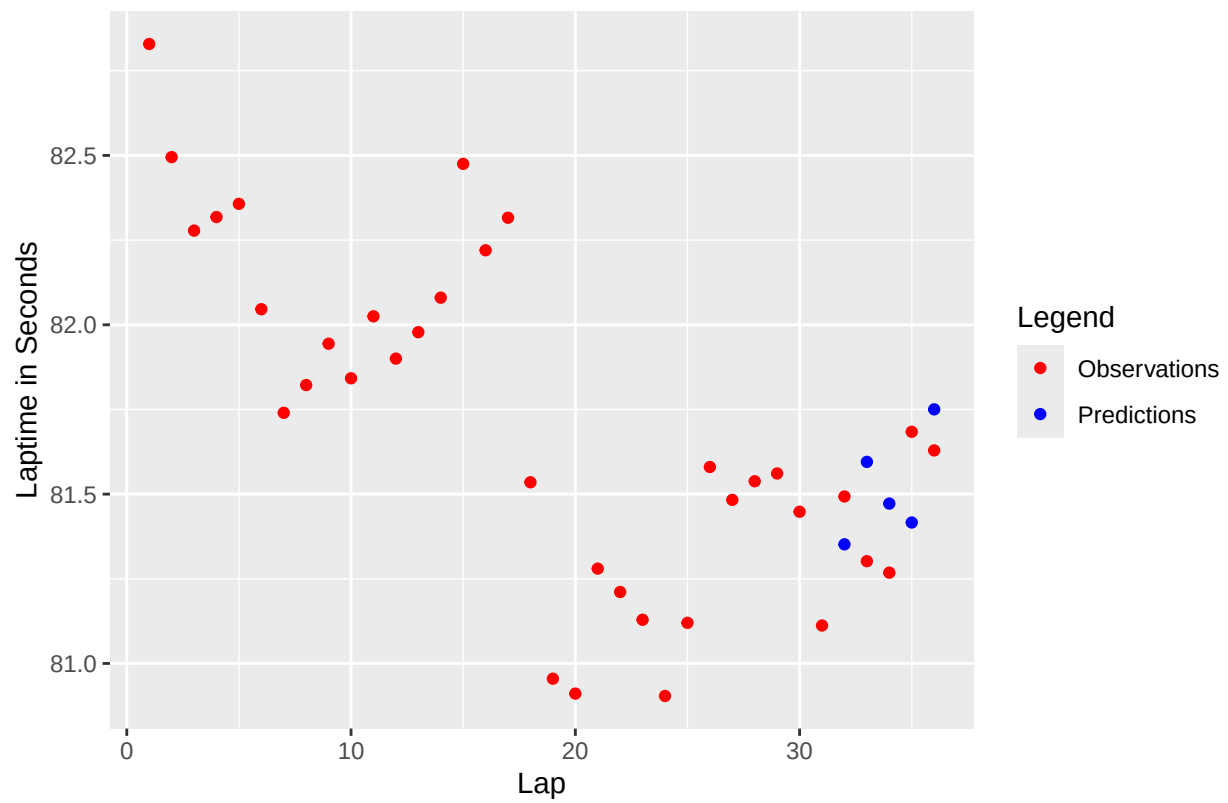
```
stan_results[[2]][[1]]
```

Scatter Plot of Lap vs Lap Times with Predictions for Stint 1



```
stan_results[[2]][[2]]
```

Scatter Plot of Lap vs Lap Times with Predictions for Stint 2



```
stan_results[[2]][[3]]
```

Scatter Plot of Lap vs Lap Times with Predictions for Stint 3

